

MANUSCRIPT SUBMISSION

CALM-Row: Navigation-Native Crop-Row Guidance from the Calibrated Box-Distribution Uncertainty of a Stock Edge Detector

Manh V. Hoang^{†1}, Thang M. Pham¹ and Nam Do¹

¹University of Engineering and Technology, Vietnam National University, Hanoi 10000, Vietnam

[†]Correspondence should be sent to manhhv87@vnu.edu.vn

Submitted: Sunday 14th June, 2026

CALM-Row: Navigation-Native Crop-Row Guidance from the Calibrated Box-Distribution Uncertainty of a Stock Edge Detector

Abstract: Vision-based navigation in row-crop agriculture estimates a guidance line whose heading and cross-track offset steer the vehicle, yet crop-row detectors are almost always reported in detection terms (mAP) and never tied to the steering error they ultimately produce. Moreover, in the standard detect-then-fit recipe a guidance line is fitted through detected row centroids with equal or heuristic weights, even though far-field boxes—which anchor the look-ahead heading—are the least reliable. We propose CALM-Row, a navigation-native, calibrated-uncertainty post-head module that runs on a *stock* lightweight detector (YOLOv8n/YOLOv10n) and introduces no new network architecture. CALM-Row reads the variance of the box-edge distribution that the Distribution Focal Loss head [1] already emits, propagates it into a per-box centroid covariance, and uses the resulting precision weights $w_i = 1/\sigma_{c_{x,i}}^2$ in a differentiable generalized-least-squares fit of the guidance line $x = ay + b$. Two auxiliary losses—a distance-aware calibration loss tying the predicted variance to realised localization error, and a line-stability loss that backpropagates the line/heading error through the weights into the existing edge logits—require no new learnable parameters, since they reuse the existing DFL distribution. Because the fit is CPU-side post-NMS math, the exported INT8/RKNN (Rockchip) or TensorRT (Jetson) graph stays a vanilla YOLO, adding zero NPU operations. We evaluate natively in navigation units (heading error in degrees, line-fit RMSE in px and cm via homography, and a static cross-track proxy in cm), together with calibration (ECE and reliability), per-seed and per-field variance, and paired significance tests against an equal-weight least-squares baseline. Headline heading error: [TODO: x.x]° for CALM-Row versus [TODO: x.x]° for the equal-weight baseline; full quantitative results are reported in the experiments.

Keywords— crop-row navigation, guidance-line estimation, calibrated uncertainty, distribution focal loss, generalized least squares, edge deployment, YOLO.

1. Introduction

Autonomous field robots are reshaping precision agriculture, replacing manual scouting and hand-steered passes with continuous, real-time operation between crop rows. The foundational competence on which most of these platforms rest is also the most deceptively simple: follow the

rows. A forward-facing camera observes the canopy, a perception module localizes the rows, and a *guidance line* is constructed that summarizes where the rows go; a steering controller then tracks this line to keep the vehicle centred between rows [2, 3]. Because the guidance line is the single interface between perception and actuation, its accuracy — expressed not as a detection score but as a heading in degrees and a cross-track offset in centimetres — directly determines whether the robot stays in the lane, damages plants, or requires human intervention.

A dominant and edge-friendly recipe for this task pairs a lightweight object detector with a downstream line fit: the detector localizes row segments as bounding boxes at successive distances, and a line is fitted through the box centroids from the bottom of the frame toward the vanishing point [4]. This family is attractive for agricultural robots because the detector can be exported to integer precision and run on commodity edge accelerators. Yet it inherits a perspective-geometry pathology that is specific to navigation. Near-field rows occupy large, texture-rich image regions and are localized precisely; far-field rows shrink to a few pixels, crowd together, and blur into soil and neighbouring vegetation, so their box edges are estimated with large and *spatially varying* uncertainty. Critically for steering, it is exactly these unreliable far-field detections that anchor the upper end of the guidance line and therefore set the look-ahead heading. A single mislocalized far box can rotate the fitted line and inject a heading error that the controller integrates forward in time [3].

We argue that the persistent difficulty here is not primarily a detection-architecture problem but a *measurement and estimation* problem, and that two gaps explain why prior work — across detection-and-line-fit [4], segmentation-and-scan [2], row-anchor regression [5], and curve regression [6] pipelines — leaves accuracy on the table. First, the optimization target and the deployed outcome are disconnected: detectors are trained and selected on mean average precision (mAP), a proxy that is never causally tied to the quantity the robot actually steers on, the guidance-line heading. Improving

mAP does not guarantee a better line, and a worse mAP does not imply worse navigation, because mAP averages over all boxes equally while navigation is dominated by a few far-field ones. Second, regardless of the perception paradigm, the guidance line is fitted with *equal* (or heuristic) weights: every point contributes identically even though the far points are an order of magnitude noisier than the near points. The estimator thus spends its precision where it is least reliable. Box-regression uncertainty has been modelled before to improve *detection* — via a Gaussian edge head [7], a KL localization loss [8], or the distributional regression head used in modern YOLO detectors [1] — but it has not been read out as a *calibrated* per-detection covariance and propagated into the guidance-line fit, where it would matter most for steering.

We close both gaps with **CALM-Row** (Calibrated Aleatoric Localization for Maize-row guidance), a post-head module that turns a stock YOLO detector into a navigation-native, uncertainty-aware crop-row guidance estimator without changing the detector graph. The key observation is that the information needed to down-weight unreliable far boxes is already present, for free, in the detector. The Distribution Focal Loss (DFL) regression head [1] predicts, per anchor, a probability distribution over R bins for each box edge $e \in \{l, t, r, b\}$; the box coordinate is the distribution mean $\mu_e = \sum_k k p_e(k)$, while its *variance* $\sigma_e^2 = \sum_k (k - \mu_e)^2 p_e(k)$ — exploited by GFLv2 [9] only as a scalar quality score, and otherwise discarded by the standard box decode — is a direct, learned, per-edge localization-uncertainty signal. Sharp distributions (confident near boxes) give small variance; flat distributions (ambiguous far boxes) give large variance. CALM-Row reads this distribution, converts it to a per-box centroid covariance by linear error propagation, and uses the resulting precision $w_i = 1/\sigma_{c_x,i}^2$ to weight a differentiable generalized-least-squares (GLS) line fit. Our contributions are as follows. **(i)** We extract a per-detection localization-uncertainty signal *for free* from the existing DFL distribution — no extra head, no extra learnable parameters in the training losses — and propagate it through closed-form error propagation to a per-box

centroid covariance ($\sigma_{c_x}^2 = (s^2/4)(\sigma_l^2 + \sigma_r^2)$, with s the stride). **(ii)** We fold this covariance into a differentiable, uncertainty-weighted GLS guidance-line fit ($x = ay + b$, parameterized along the image rows to avoid the vertical-line degeneracy of near-vertical crops), whose closed-form normal equations and parameter covariance also yield a principled heading $\theta = \arctan a$ and an associated heading uncertainty. **(iii)** We train the detector with two auxiliary, parameter-free losses that operate purely on the DFL distribution: a distance-aware calibration loss (DDC), a Gaussian negative-log-likelihood that ties the predicted centroid variance to the realised squared centroid error on matched anchors so the uncertainty becomes *calibrated* rather than merely relative, and a line-stability loss (LSL) whose gradients flow through the weights w_i into the DFL logits, teaching the network to down-weight unreliable far boxes for a stabler look-ahead line. **(iv)** We design an evaluation in navigation units — heading error (degrees) as the controller-independent primary metric, line-fit RMSE (px and cm via a stated homography), and a static geometric cross-track proxy at the look-ahead row (a closed-loop pure-pursuit simulation over video sequences is left to future work) — alongside calibration (ECE, reliability) and edge latency, with generalization measured in-domain on CRDLD and as cross-dataset transfer to CRBD, and statistical claims supported by multiple seeds, paired significance tests, and effect sizes. An illustrative synthetic mechanism check is given in Section 5.6; all dataset-level navigation, calibration, mAP and latency numbers are reported only after the experiments of Section 5 are run.

Because all of CALM-Row’s computation is post-head CPU arithmetic over post-NMS boxes, it is detector- and neck-agnostic and adds *zero* new operators to the on-device graph: the exported INT8 model — whether targeting an RKNN (Rockchip) NPU or TensorRT (Jetson) — remains a vanilla YOLO, and the calibration and line-stability losses introduce no new learnable parameters during training. CALM-Row therefore upgrades the accuracy of the guidance line, and connects it to the navigation outcome, at no inference-time cost on the edge hardware that agricultural robots actually

99 deploy.

100 2. Related work

101 We organise prior art around the three threads that bound our contribution: vision-based crop-row
102 navigation (the application and its evaluation conventions), uncertainty modelling in object detection
103 (the signal we exploit), and multi-scale feature fusion (an architectural axis we treat as orthogonal).
104 After each thread we state explicitly what it leaves unaddressed, so that the gap CALM-Row fills is
105 unambiguous.

106 2.1. Vision-based navigation in row-crop agriculture

107 Crop-row guidance from a forward-facing camera has converged on a small number of output
108 paradigms, which we group by how the guidance line is produced.

109 **Detection plus centroid plus line fit.** The de-facto recipe localises row segments or plant
110 clusters with an object detector and fits a single guidance (centre) line through the box centroids.
111 Representative is the YOLOX-based maize-row system of Diao et al. [4], which detects row patches,
112 takes their centroids, and fits a centreline by least squares, reporting angular error. This is the
113 family our stock YOLOv8n/YOLOv10n detector belongs to. Earlier and adjacent crop-row pipelines
114 similarly localise vegetation and then fit a line or band through extracted points [2]. The defining
115 limitation for our purposes is that the line fit treats every centroid as equally trustworthy: each point
116 contributes with unit (or hand-tuned heuristic) weight, even though the far-field boxes that dominate
117 the look-ahead heading are precisely the smallest and least reliable detections.

Segmentation plus geometric scan. A second paradigm segments the crop/soil mask and recovers the row geometry by a triangle or scan-line search over the mask. The de Silva et al. line of work [2, 3] validates this approach across many field conditions and, importantly for this paper, reports results in navigation units (cross-track and heading error) on the CRDLD benchmark [2]; we adopt its mask-derived centreline as our detector-independent ground-truth line. We stress that this evaluation line is computed from masks, not from detections, precisely to avoid circularity in the weighted-versus-unweighted comparison. The geometric scan itself is a hard, non-differentiable search and carries no per-point reliability estimate.

Row-anchor and lane-style regression. A third paradigm ports the row-anchor formulation from lane detection—UFLD [10] and LaneATT [11]—to agricultural rows, as in ALNet [5] and row-column attention [12]. These methods classify or regress a horizontal position per image row and connect the selected anchors into a curve. They are accurate but architecture-specific, and the per-anchor confidence (when present) is a classification score, not a calibrated geometric covariance on the recovered line.

Polynomial / curve regression. A fourth paradigm regresses the row directly as a parametric curve; RowDetr’s per-row polynomial regression with INT8 edge deployment is representative [6]. The curve parameters are produced end-to-end, but again with no per-segment uncertainty propagated into the curve estimate.

What this thread leaves open. Two observations recur across all four paradigms and motivate our work. First, the literature evaluates in *navigation units*—cross-track error (cm), heading error (deg), and intervention rate—rather than detection mAP, because a detector that scores well on mAP can still yield a poor guidance line, and vice versa. A method that aims to improve navigation must

therefore be measured there. Second, and more specific to our contribution: *regardless of paradigm, the guidance line is fit from points or curves with equal or heuristic weights* — intensity-weighted crop-row regression is itself two decades old [13], but those weights are a pixel-intensity heuristic. None of these systems carry a per-detection, calibrated reliability estimate into the geometric line estimator, even though far-field predictions are systematically the least reliable. This is the gap CALM-Row addresses.

2.2. Uncertainty in object detection

A parallel line of work models the uncertainty of bounding-box regression. KL-Loss [8] predicts a per-coordinate variance under a Gaussian assumption and uses it to weight box coordinates during variance-voting NMS. Gaussian-YOLO [7] attaches a predicted localisation uncertainty to each YOLO box and uses it to down-weight uncertain detections at inference. Most relevant to us, Generalized Focal Loss and its Distribution Focal Loss (DFL) head [1] replace the single-value box regression with a learned discrete distribution over a set of bins for each box edge; this is the regression head used by modern stock YOLO detectors [14–16], including the YOLOv8n/YOLOv10n we build on. For each anchor and edge $e \in \{l, t, r, b\}$ this head produces a softmax distribution $p_e(k)$ over bins $k = 0, \dots, R - 1$, whose mean $\mu_e = \sum_k k p_e(k)$ is the predicted edge offset and whose variance $\sigma_e^2 = \sum_k (k - \mu_e)^2 p_e(k)$ already encodes the network’s spread (sharp distributions for confident edges, flat distributions for ambiguous ones).

What this thread leaves open. In every case above the predicted spread is consumed *within the detector*: KL-Loss and Gaussian-YOLO use variance to reweight or suppress boxes for better detection metrics. Closest to us, Generalized Focal Loss V2 [9] explicitly reads the *shape* of the DFL distribution (its top- k values and mean), but maps it to a scalar localisation-*quality* score used *inside*

the detector for classification/NMS ranking; the distribution is otherwise collapsed to its expectation μ_e to produce the box coordinate. What remains open is to read the distribution’s *second moment* as a *metric* per-detection covariance (in pixels), calibrate it against realised error, and propagate it as the precision matrix of a *downstream differentiable geometric estimator outside the detector* — a role a unitless ranking score cannot serve. CALM-Row does exactly this: it converts the existing per-edge DFL variance into a per-box centroid covariance by linear error propagation, calibrates it against realised localisation error, and uses its inverse as the precision weight in a differentiable generalized-least-squares guidance-line fit. Because the signal is already produced by the stock head, no new operator runs on the feature maps or the NPU—in contrast to KL-Loss/Gaussian-YOLO, which add a dedicated uncertainty branch. Where such a dedicated head is runnable, we report it as an exploratory comparison. Our differentiable weighted fit is structurally related to end-to-end differentiable least-squares lane fitting [17]; the difference is the *source* of the weights — a learned, uncalibrated attention map from an extra head there, versus a calibrated metric covariance read for free from the existing DFL head here.

2.3. Multi-scale feature fusion (orthogonal)

For completeness we note the large body of work on the detector neck. We treat the detector neck as orthogonal to our contribution and make this explicit. Feature Pyramid Networks [18] and the bottom-up PANet extension [19] established top-down/bottom-up multi-scale fusion; EfficientDet’s BiFPN [20] learns scalar per-edge importance weights; and *Adaptively Spatial Feature Fusion* (ASFF) [21] learns spatially-varying per-pixel weights over pyramid levels. These methods improve the *features* a detector computes. CALM-Row, by contrast, is a post-head, CPU-side module over post-NMS boxes and does not modify the neck, the backbone, or the exported graph. It is therefore compatible with any of these necks, and we verify in our experiments that its benefit is not

neck-specific by repeating the controlled comparison on a stock PAN neck [19] and on an ASFF neck [21]. The choice of neck is consequently *not* a claim of this paper.

2.4. Positioning of CALM-Row

In summary, CALM-Row sits at the intersection of the three threads above and is defined by what it does *not* do: it adds no neck or architectural component (Section 2.3), it adds no new detector branch or NPU operator (Section 2.2), and it does not introduce a new line-detection architecture (Section 2.1). Its single contribution is to close the loop between the uncertainty the DFL head [1] already produces and the equal-weight line fit that crop-row navigation systems [2, 4–6] currently use: a calibrated, uncertainty-weighted, navigation-native guidance-line estimator trained and evaluated on the navigation task itself.

3. CALM-Row: calibrated, uncertainty-weighted crop-row guidance

We do *not* propose a new backbone, neck, or detection head. CALM-Row is a post-head module that sits on top of an unmodified, edge-deployable stock lightweight YOLO detector (we use the YOLOv8n and YOLOv10n nano variants of the YOLO family [14, 16]) and turns its existing box-regression *distribution* into a calibrated per-detection localization covariance, which is then used as the precision weight in a differentiable generalized-least-squares (GLS) guidance-line fit. Crucially, every operation introduced below is read out from quantities the detector already computes; the two training losses add *no new learnable parameters* and *no new operators on the feature maps*, so the exported INT8/RKNN or TensorRT graph remains a vanilla YOLO (Section 3.6). Figure 1 summarizes the pipeline.

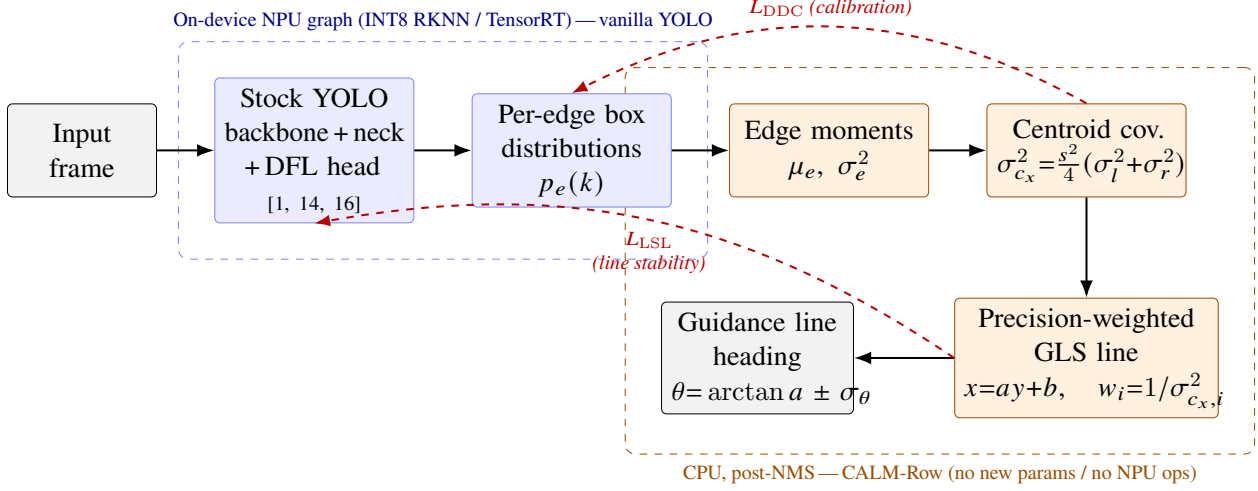


Figure 1: CALM-Row pipeline. A *stock* YOLO detector (blue) emits, per box edge $e \in \{l, t, r, b\}$, a Distribution-Focal-Loss distribution $p_e(k)$ [1]; its mean is the usual box coordinate while its *variance* σ_e^2 is read out (orange, CPU, post-NMS) and propagated to a per-box centroid covariance, then used as the precision weight w_i in a differentiable generalized-least-squares fit of the guidance line $x = ay + b$ and its heading θ . Two training-only losses (dashed red) make the variance *calibrated* (L_{DDC}) and the fitted line *stable* (L_{LSL}), backpropagating through the weights into the existing edge logits. No new learnable parameters and no new operators on the feature maps are introduced, so the exported edge graph stays a vanilla YOLO.

3.1. Problem setup and the detect $\rightarrow$ centroid $\rightarrow$ line pipeline

Given a forward-facing field image, the detector returns a set of crop-row segment boxes after non-maximum suppression (NMS). For box i we take its centroid $(c_{x,i}, c_{y,i})$ in pixels, where c_y is the image row. Following the de-facto detection-and-line-fit recipe for row-crop navigation [4], a single guidance line is fit through the centroids. We parameterise the line as

$$x = ay + b, \quad (1)$$

i.e. image column x as a function of image row y . This orientation is deliberate: crop rows viewed head-on are near-vertical, so a conventional $y = mx + c$ fit becomes ill-conditioned as the slope

diverges; expressing x as a function of y removes this vertical-line degeneracy. The heading angle is $\theta = \arctan(a)$, reported directly in degrees as the controller-independent navigation quantity, and the cross-track offset follows from (a, b) evaluated at the look-ahead row.

The departure from the standard recipe is *how* the line is fit. Far-field boxes (top-of-frame, small scale) anchor the look-ahead heading yet are the least reliable localizations; an equal-weight fit lets their error dominate exactly where navigation is most sensitive. CALM-Row instead fits the line by precision-weighting each centroid with a *calibrated* estimate of its own localization variance, read directly from the detector’s box-edge distribution.

3.2. Uncertainty readout from the DFL distribution

Modern YOLO heads regress each box edge with a Distribution Focal Loss (DFL) head [1]: rather than a single offset, the network emits, per anchor, a softmax distribution over $R = \text{reg_max}$ bins for each of the four edges $e \in \{l, t, r, b\}$ (left, top, right, bottom), in stride units. Let $p_e(k)$ denote that distribution over bins $k = 0, \dots, R - 1$. The DFL integral (the value the detector already uses to decode the box) is the distribution *mean*, and CALM-Row additionally reads its *variance*, which the standard pipeline simply discards:

$$\mu_e = \sum_{k=0}^{R-1} k p_e(k), \quad \sigma_e^2 = \sum_{k=0}^{R-1} (k - \mu_e)^2 p_e(k). \quad (2)$$

A sharp (confident) edge distribution yields a small σ_e^2 ; a flat (ambiguous) one yields a large σ_e^2 . Both moments are in stride units and become pixels on multiplication by the anchor stride s .

Treating the four edges as independent and propagating their variance to the centroid by first-order (linear) error propagation gives the per-box centroid covariance in pixels². Since c_x depends on the

232 left/right edges and c_y on the top/bottom edges, the diagonal centroid covariance is

$$\sigma_{c_x}^2 = \frac{s^2}{4}(\sigma_l^2 + \sigma_r^2), \quad \sigma_{c_y}^2 = \frac{s^2}{4}(\sigma_t^2 + \sigma_b^2). \quad (3)$$

233 Equations (2)–(3) are exactly the readout implemented in the reference module (`dfl_mean_var` and
 234 `centroid_cov_from_edges`). Because the guidance line (1) regresses x on y , the relevant per-box
 235 precision is

$$w_i = \frac{1}{\sigma_{c_x,i}^2 + \epsilon}, \quad (4)$$

236 with a small ϵ for numerical stability. No parameters are introduced: w_i is a deterministic function
 237 of the DFL logits the head already produces. We contrast this with detection-uncertainty methods
 238 that *add* a dedicated variance head (KL-Loss [8], Gaussian-YOLO [7]); here the uncertainty is free,
 239 and we make it *calibrated* and *navigation-aware* through the two losses below.

240 3.3. Distance-aware calibration loss L_{DDC}

241 The raw DFL variance is a sharpness statistic, not a calibrated error bar: nothing ties its magnitude
 242 to the *realised* localization error. The distance-aware calibration loss (L_{DDC}) supplies this tie as a
 243 heteroscedastic-aleatoric Gaussian negative log-likelihood (NLL) [22] over matched (foreground)
 244 anchors, treating each predicted centroid coordinate as a Gaussian whose variance is the propagated
 245 σ_c^2 of Equation (3) and whose target is the matched ground-truth centroid. For a matched anchor
 246 with predicted centroid $(\hat{c}_x, \hat{c}_y)$ and ground-truth centroid (c_x^*, c_y^*) ,

$$L_{\text{DDC}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \frac{1}{2} \sum_{u \in \{x,y\}} \left[\frac{(\hat{c}_{u,i} - c_{u,i}^*)^2}{\sigma_{c_{u,i}}^2 + \epsilon} + \log(\sigma_{c_{u,i}}^2 + \epsilon) \right], \quad (5)$$

where $\mathcal{M}$ is the set of matched anchors from the detector’s own assigner. The data-fidelity term penalises over-confident (too-small) variances on boxes whose centroid is actually far from the target, while the $\log \sigma^2$ term penalises uniformly inflating the variance; their balance makes σ_c^2 a calibrated predictor of localization error. Because far/small boxes incur larger realised errors, the loss naturally drives their variance up — the “distance-aware” behaviour — without ever using a depth label. The gradient flows back into the existing DFL logits, so the detector *learns to express honest uncertainty* in its distribution shape. After training (and after INT8 quantization), an optional per-class affine recalibrator may be re-fit on a small validation set for reporting; it is an inference-time convenience and is not part of the trained graph.

3.4. Differentiable precision-weighted GLS line fit and the line-stability loss

Closed-form GLS fit. Given $N \geq 2$ centroids with precisions w_i , fitting line (1) is the weighted (generalized) least-squares problem

$$(\hat{a}, \hat{b}) = \arg \min_{a,b} \sum_{i=1}^N w_i (x_i - a y_i - b)^2, \quad (6)$$

which has the closed-form solution of the 2×2 normal equations

$$\underbrace{\begin{bmatrix} \sum_i w_i y_i^2 & \sum_i w_i y_i \\ \sum_i w_i y_i & \sum_i w_i \end{bmatrix}}_{\mathbf{X}^T \mathbf{W} \mathbf{X}} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \sum_i w_i x_i y_i \\ \sum_i w_i x_i \end{bmatrix}. \quad (7)$$

The two diagonal entries differ by several orders of magnitude (one scales with y^2 , the other with N), so the implementation adds a per-diagonal ridge scaled to each entry’s own magnitude before solving, which keeps the small intercept entry well-conditioned without biasing the slope. The

entire fit (7) is composed of differentiable reductions and a 2×2 solve, so gradients propagate from $(\hat{a}, \hat{b})$ back through the weights w_i into the DFL logits; the network can therefore *learn to down-weight unreliable far boxes* so that the fitted line is stable. The parameter covariance and heading uncertainty are obtained in closed form from the same matrix,

$$\text{Cov} \begin{bmatrix} a \\ b \end{bmatrix} = s_0^2 (\mathbf{X}^\top \mathbf{W} \mathbf{X})^{-1}, \quad s_0^2 = \frac{1}{N-2} \sum_{i=1}^N w_i r_i^2, \quad \sigma_\theta^2 = \frac{\text{Var}(a)}{(1+a^2)^2}, \quad (8)$$

where $r_i = x_i - \hat{a}y_i - \hat{b}$ are the residuals and s_0^2 is the empirical variance factor that keeps the heading interval honest when the weights are calibrated-but-not-exact; σ_θ follows from $\theta = \arctan(a)$ by error propagation. This yields a predicted heading *and* its uncertainty, enabling the heading-level reliability check used in evaluation.

Line-stability loss L_{LSL} . L_{DDC} makes the per-box variances trustworthy; the line-stability loss (L_{LSL}) makes the *resulting line* accurate where it matters. For each training image we form two lines: the predicted line $(\hat{a}, \hat{b})$ from the weighted GLS fit over the detector’s predicted centroids and weights, and a ground-truth training line $(a^\star, b^\star)$ fit *unweighted* through the ground-truth box centroids of that image. We then penalise the discrepancy between the two lines, sampled over y extended to the far look-ahead row y_{far} so that far-field stability is explicitly supervised, plus a heading term:

$$L_{\text{LSL}} = \sqrt{\frac{1}{|\mathcal{Y}|} \sum_{y \in \mathcal{Y}} \left[(\hat{a}y + \hat{b}) - (a^\star y + b^\star) \right]^2} + \lambda_\theta |\arctan(\hat{a}) - \arctan(a^\star)|, \quad (9)$$

where $\mathcal{Y}$ is a set of rows sampled from the observed span down to y_{far} and λ_θ balances the column-RMSE and heading terms. The ground-truth training line is fit through ground-truth box *centroids* and is therefore independent of the detector and of the weighting scheme; this avoids circularity for

the weighted-vs-unweighted comparison, which is adjudicated separately at evaluation against a *mask-derived* ground-truth line (the detector-independent centerline). Gradients from Equation (9) flow through $\hat{a}$ and $\hat{b}$ into w_i and onward into the DFL logits, closing the loop: the detector is trained so that its uncertainty-weighted line matches the true row, with far boxes down-weighted when unreliable.

3.5. Total objective

CALM-Row is trained end-to-end by augmenting the standard YOLO detection loss L_{det} (box CIoU + DFL + classification [1, 14]) with the two auxiliary terms:

$$L = L_{\text{det}} + \alpha L_{\text{DDC}} + \beta L_{\text{LSL}}, \quad (10)$$

with non-negative scalars α, β (the calibration and line gains). Setting $\alpha = \beta = 0$ recovers the stock detector exactly, which defines the controlled baseline; the two terms are ablated independently in Section 5 to attribute the gain. Neither term introduces learnable parameters: both are computed from the existing DFL distribution and the detector’s own assigner, so L_{DDC} and L_{LSL} change *what the network learns to predict*, not *what it computes at inference*.

3.6. Inference and edge deployment

At inference, the detector runs unmodified and produces post-NMS boxes together with the per-box edge logits it already emits. CALM-Row then executes purely on the CPU over these post-NMS detections: it reads the DFL variance (2), propagates it to the centroid covariance (3) and precisions (4), and solves the 2×2 GLS system (7) to return the guidance line $(\hat{a}, \hat{b})$, the heading $\theta = \arctan(\hat{a})$, and its uncertainty σ_θ (8). This is a handful of reductions and a 2×2 solve over the

(few) detected boxes, with negligible cost relative to the detector forward pass.

The decisive deployment property is that *nothing* CALM-Row adds touches the feature maps or the NPU/accelerator graph. All CALM-Row math is post-head, post-NMS CPU arithmetic on quantities the detector already outputs, so it introduces *zero new accelerator operators* and *zero new learnable parameters in the exported model*. The detector graph exported to INT8 for a Rockchip NPU via RKNN, or to FP16/INT8 for a Jetson via TensorRT, is bit-for-bit a vanilla YOLO; the edge runtime is therefore unaffected, and the calibrated-uncertainty guidance line is obtained for free on the host CPU. Because the DFL distribution is itself quantized, we re-fit the optional inference-time recalibrator on a small validation set after INT8 conversion so that calibration is preserved across quantization (evaluated in Section 5). An illustrative synthetic mechanism check is given in Section 5.6; the empirical navigation evaluation on real data is reported in Section 5.

4. Experimental setup

This section specifies the protocol used to evaluate CALM-Row. The evaluation is deliberately *navigation-native*: the headline quantities are heading (angular) error in degrees and guidance-line error in centimetres, with detection mAP reported only as a secondary diagnostic. All comparisons are organised around a single controlled axis — how the guidance line is fitted through the detector’s box centroids — so that any measured difference is attributable to the precision weighting and calibration introduced by CALM-Row rather than to a different backbone, neck, or training recipe. No new operator is introduced on the detector graph: CALM-Row is post-head CPU arithmetic over the variance of the existing distribution-focal-loss (DFL) box-edge head [1], so the exported edge graph is a vanilla YOLO. Because no experiments have yet been run, every quantitative cell below is a placeholder (“--”); the only concrete figure stated anywhere is an explicitly labelled synthetic

sanity check of the fitting mechanism (Section 4.8).

4.1. Datasets and ground truth

Detection labels from line GT. Neither benchmark provides bounding boxes: CRDLD ground truth is a binary mask of the crop-row *centrelines* and CRBD provides parametric row annotations. As CALM-Row runs on a stock DFL box detector, we synthesise detection labels from the line GT: from each CRDLD mask we extract the central (navigation) crop row and place a short sequence of boxes along it at successive distances (ordinal class near . . . far, perspective-scaled size). The box labels are clean, yet the detector still learns image difficulty, so the DFL distribution is naturally flatter for far rows. The GT guidance line used for *evaluation* is a robust fit through the mask/annotation centreline (Section 4.3) and is detector-independent, so the weighted-versus-unweighted comparison is not circular.

In-domain training and test: CRDLD. The Crop Row Detection dataset of De Silva et al. [2] (512×512 ; train/validation/test = 1250/250/430) trains the detector and serves as the in-domain test set on which the confirmatory hypotheses (Section 4.6) are evaluated.

Cross-dataset generalisation: CRBD. The Crop Row Benchmark Dataset [23] (320×240 ; a different camera, crop and resolution; no training split) is a held-out *cross-dataset* transfer test: the CRDLD-trained detector is evaluated on CRBD without retraining — a stronger generalisation probe than a within-dataset split. The CRDLD download used here carries no per-field metadata, so a within-CRDLD leave-one-field-out protocol is unavailable; cross-dataset transfer to CRBD takes its place.

The CRDLD train/validation/test splits are disjoint, and CRBD is an entirely separate dataset, so memorisation cannot inflate the transfer result. A Data and Code Availability statement accompanies the manuscript.

Table 1 summarises the role of each dataset.

Table 1: Datasets and their role. Boxes are synthesised from the line GT (Section 4.1); the evaluation GT line is annotation-derived and detector-independent. Results to be completed.

Dataset	Role	GT guidance line	cm scale?
CRDLD [2] (512 ² ; 1250/250/430)	Train detector + in-domain test	line-mask → central row	–
CRBD [23] (320×240; 283)	Cross-dataset transfer test	annotation → central row	–

4.2. Pixel-to-centimetre calibration

Because “cm” is meaningless without a metric anchor, we calibrate image-to-ground scale explicitly and report centimetres only where it is justified. Where camera intrinsics and extrinsics (or a checkerboard reference) are available, we recover the image-to-ground homography H under an explicit *flat-ground assumption* and report it. Otherwise, metric scale is anchored from a *known inter-row spacing* (for instance the nominal 0.75 m spacing of maize rows, used purely as a calibration constant, not as a measured outcome); where neither is available we report errors in *pixels* and in pixels normalised by image width, and do not invent a centimetre value.

Flat ground is stated as an explicit limitation, accompanied by an error budget: a camera-pitch perturbation θ propagates to a look-ahead-distance error that grows with range, so we tabulate the induced centimetre error at the look-ahead row as a function of θ . This makes the conditions under which the centimetre numbers are trustworthy auditable.

4.3. Frozen ground-truth guidance-line extraction

The ground-truth (GT) guidance line used for *evaluation* is derived from segmentation masks and is therefore *detector-independent*. The extraction algorithm is frozen as follows. For each crop-row mask, we compute the row-wise (per image row y) median of foreground x , fit a robust

(Huber) regression to these per-row medians, and select as the navigation row the one nearest the image-centre-bottom; gap and occlusion handling are defined as part of the frozen procedure. The resulting line is expressed in the same $x = a y + b$ parameterisation used by CALM-Row, where x is predicted as a function of row y to avoid the vertical-line degeneracy of near-vertical crop rows; heading is $\theta = \arctan(a)$.

To validate the auto-extracted GT, we cross-check it against a *human-annotated subset* of at least 100 frames and report inter-annotator agreement in both degrees and centimetres (Table 2). Crucially, the evaluation GT is mask-derived, *not* an unweighted least-squares fit through GT box centroids: using a centroid-fit line as the evaluation target would be circular for a study whose central claim is that uncertainty-weighted fitting beats unweighted fitting. The unweighted centroid-fit line is used only as a *training* target inside the line-stability loss, where the detector-independent GT-box centroids make it non-circular with respect to the weighting claim, which is itself evaluated separately against the mask-derived line.

Table 2: Frozen GT-line extraction: agreement between the automatic mask-derived line and human annotation on the cross-check subset (≥ 100 frames). Results to be completed.

Quantity	Heading ($^{\circ}$)	Cross-track (cm)
Auto vs. human (median $ \Delta $)	–	–
Inter-annotator (median $ \Delta $)	–	–

4.4. The A/B evaluation protocol

The study rests on one controlled comparison: *does turning the guidance-line fit from equal-weight into DFL-uncertainty-weighted (CALM-Row) reduce navigation error?* To attribute the effect cleanly we separate detector *training* from line *readout* at inference, defining the arms in Table 3. CALM-Row converts the variance of each box-edge DFL distribution into a per-box centroid covariance and uses its inverse as the precision weight $w_i = 1/\sigma_{c_x,i}^2$ in a differentiable

generalised-least-squares (GLS) fit of $x = a y + b$, with heading variance read from the WLS parameter covariance $s_0^2(X^\top W X)^{-1}$, where $s_0^2 = \sum_i w_i r_i^2 / (N - 2)$ is the empirical variance factor over the N fitted boxes.

Table 3: A/B arms. A, A' and B0 share one stock checkpoint and differ only in the CPU-side readout, so they are compared paired per frame; B uses its own checkpoint trained with the CALM-Row auxiliary losses. Results to be completed.

Arm	Detector training	Line readout at eval	Isolates
A (baseline)	stock losses	equal-weight LS through centroids [4]	reference
A'	stock losses	RANSAC LS	robust baseline
B0 (readout only)	stock losses	precision-weighted GLS, <i>raw</i> DFL variance	gain from weighting alone
B (CALM-Row)	+ DDC + LSL aux losses	precision-weighted GLS, calibrated variance	gain from calibrated training
B-cal, B-line	one aux loss off	weighted GLS	which aux loss carries the gain

Design points. Arms A, A' and B0 share a single trained checkpoint (the stock detector) and differ only in the post-NMS line readout; their comparison is therefore *paired per test frame* and free of training noise. Arm B requires its own checkpoint trained with the two auxiliary losses. The decomposition is reported honestly: *B0 vs. A* answers “is the free DFL variance already useful?”, and *B vs. B0* answers “does the CALM-Row training add more?”. The auxiliary losses are the distance-aware calibration loss (DDC, a Gaussian negative-log-likelihood tying σ_c^2 to realised squared centroid error on matched anchors) and the line-stability loss (LSL, an RMSE-to-the-far-look-ahead-row plus heading term between the GLS line over predicted centroids/weights and the GT line); both reuse the existing DFL distribution and add *no new learnable parameters*. The total objective is $L = L_{\text{det}} + \alpha L_{\text{DDC}} + \beta L_{\text{LSL}}$, with L_{det} the standard YOLO box-CIoU + DFL + cls loss.

Detectors and necks. The A/B axis is run on two Nano-deployable stock detectors, YOLOv8n [14] and YOLOv10n [16]. To show that the gain is not architecture-specific, we repeat the A/B comparison on at least two *published* necks — the stock PANet fusion [19], ASFF [21], and BiFPN [20] — as a neck-agnosticity check. No bespoke neck is introduced; this paper makes no neck-architecture claim.

Evaluation harness. A single script (`eval_guidance.py`) takes a checkpoint, a test split, and a readout choice (`equalLS` / `ransac` / `calm` / `qual` / `oracle`) and emits per-frame navigation metrics plus a per-frame CSV for paired statistics. For each image it runs the detector while retaining per-box *edge logits* (pre-DFL-integral), boxes and strides; builds the guidance line with the chosen readout (equal-weight / RANSAC unweighted, or the precision-weighted GLS of CALM-Row); loads the frozen mask-derived GT line (Section 4.3); and computes, sampled to the look-ahead row, line-fit RMSE (px and cm), heading error (deg), and the static geometric cross-track proxy (cm) at the look-ahead row (Section 4.5). It also logs, per box, predicted σ versus realised |centroid error| for calibration, stratified by an independent range proxy. Writing one CSV row per frame lets the shared-checkpoint arms be compared paired.

4.5. Metrics

Primary (navigation). Heading / angular error (deg) is the main, controller-independent headline number. Line-fit RMSE is reported in pixels (and cm where Section 4.2 permits), evaluated *including the look-ahead row*, not only the observed span. Edge throughput (FPS), latency, parameters and GFLOPs are reported on the named devices of Section 4.9.

Secondary. Cross-track is reported as a *static geometric proxy*: the lateral offset (px, and cm where the homography permits) between the predicted and ground-truth guidance lines at the look-ahead row. A full closed-loop pure-pursuit / kinematic-bicycle simulation (fixed wheelbase, speed, look-ahead L_d , control rate, and perception latency injected from the measured edge FPS, with the *same frozen controller* for every arm) is specified in the experiment plan but *left to future work*, as it requires sequential field video rather than the single-frame test sets used here. Detection mAP@50 and mAP@50–95 are reported with variance as secondary diagnostics.

Calibration. We report expected calibration error (ECE) and reliability diagrams of predicted σ

versus *realised* localisation error, stratified against an independent range proxy (image row y under flat ground, or box scale in px) by *continuous* regression of σ on the proxy rather than treating the categorical distance bins as range labels. Beyond per-box ECE, we report heading-level reliability (whether predicted heading σ covers realised heading error at the 68 % and 95 % levels). Calibration is measured *before and after* INT8 quantisation, re-fitting the optional inference-time variance scaler post-INT8, to test the claim that calibration survives quantisation.

The main A/B results are reported in Table 5.

4.6. Statistical protocol

We *pre-register* the confirmatory hypotheses (**H1**) CALM-Row beats equal-weight LS on heading error (the core A/B) and (**H3**) the H1 heading gain holds on published necks [19–21]. As an *exploratory* (non-confirmatory) comparison (**H2**), where a separate Gaussian/KL uncertainty head [7, 8] is runnable we report whether the free DFL variance is competitive on calibration (ECE) at lower parameter count and latency. All other analyses are exploratory.

Confirmatory comparisons use ≥ 5 seeds; exhaustive ablations use 3 seeds (stated per table). Because n is small for normality, significance uses the *Wilcoxon signed-rank* test (paired per frame and per seed), not a t -test, with *Holm–Bonferroni* correction across the confirmatory family; adjusted p -values are reported. We report effect sizes (Cliff’s δ) with 95 % bootstrap confidence intervals, not merely $p < 0.05$. Variance is decomposed into across-seed (optimisation) and cross-dataset (CRDLD→CRBD generalisation) components; the unit of generalisation is the dataset, not the seed. Any claim whose Δ is smaller than its standard deviation is downgraded in wording.

Pre-registered pass/fail. H1 passes iff B beats A on heading error with Holm-adjusted Wilcoxon $p < 0.05$ and Cliff’s $\delta \geq 0.33$ (significant *and* non-trivial). B0 vs. A is reported regardless: if weighting alone already wins it is a cheap, clean result; if only B wins, the training is doing the

work. The far-field analysis (the far rows of Table 6) repeats H1 on the held-out far rows only, where the effect is expected to be larger. Paired significance for H1 is reported in Table 6.

4.7. System-level positioning

Separately from the controlled A/B grid, we position CALM-Row against published crop-row methods of *different* architectures on the same test split, the same mask-derived GT line, the same centimetre metric, and the same edge device: the de Silva segmentation-plus-geometric-scan pipeline [2], RowDetr’s polynomial regression [6], and the row-anchor / attention family [5] (where each is runnable). We state explicitly that this comparison is *not* controlled — the systems differ in architecture and output paradigm — so the takeaway is the navigation-accuracy / edge-efficiency frontier, not a like-for-like win.

4.8. Synthetic mechanism check

The synthetic, mechanism-level sanity check of the estimator (illustrative only, not a dataset result) is reported once in Section 5.6. The real far-field evidence comes from the held-out far rows of Section 4.6 measured against the mask-derived GT line.

4.9. Edge deployment

We report *two* edge platforms in one latency table to avoid the common Rockchip-versus-Jetson confusion: TensorRT (FP16 and INT8) on an NVIDIA Jetson board, and RKNN INT8 on a Rockchip NPU board (e.g. RK3588), which matches the deployment target. For each we report latency mean $\pm$ std and P95, throughput, power (W) and energy per frame, thermal/throttling state, batch

size, and input resolution (Table 4). We verify that CALM-Row adds *zero* NPU operations — it is post-NMS CPU arithmetic over the existing DFL logits — so the exported detector graph is a vanilla YOLO, and we re-derive both the accuracy/efficiency and the navigation-error/efficiency Pareto fronts under deployment precision. Structured pruning [24] is available as an optional efficiency lever and, if applied, is reported on the same axes.

Table 4: Edge deployment on the two named platforms. CALM-Row adds zero NPU ops; the exported graph is a vanilla YOLO. Results to be completed.

Platform	Precision	Latency mean $\pm$ std (ms)	P95 (ms)	FPS	Power (W)	Energy/frame (mJ)
Jetson (TensorRT)	FP16	—	—	—	—	—
Jetson (TensorRT)	INT8	—	—	—	—	—
Rockchip (RKNN)	INT8	—	—	—	—	—

5. Results

This section reports the empirical evaluation of CALM-Row. *No dataset-level numbers are filled in yet*: every quantitative cell is a placeholder (“--” in tables, “[TODO]” in prose) to be completed once the pre-registered runs of the A/B protocol (Section 4.4; see also the experiment plan) have finished. The table structures, comparison arms, metrics, and statistical tests below are fixed in advance so that the analysis is confirmatory rather than exploratory. The only concrete number stated anywhere in this paper is a synthetic, mechanism-level sanity check that is *not* a dataset result and is labelled as illustrative (Section 5.6).

We restate the canonical evaluation design so the tables are self-contained. Four guidance-line readout arms are compared. Arms **A** (equal-weight least squares through box centroids, the Diao-style recipe [4]), **A'** (RANSAC least squares), and **B0** (precision-weighted GLS using the *raw* variance of the existing DFL box-edge distribution [1]) share a single stock detector checkpoint and differ only in the CPU-side readout, so their comparison is paired per frame and free of training noise. Arm **B**

(full CALM-Row) is trained with the distance-aware calibration loss L_{DDC} and the line-stability loss L_{LSL} and uses the calibrated precision-weighted GLS readout with weights $w_i = 1/\sigma_{c_x,i}^2$. The decomposition is deliberate: **B0 vs. A** isolates the gain from *weighting alone* (free, no extra training), while **B vs. B0** isolates the additional gain from *training the detector to be calibrated and line-aware*. The primary metric is heading error (deg), which is controller-independent; line-fit RMSE (px and, where the px→cm homography of Section 4.2 applies, cm) and closed-loop cross-track error (cm) are secondary; calibration (ECE), detection (mAP@50, mAP@50–95), and edge latency/FPS support the deployment argument.

5.1. Main A/B comparison per detector (T1)

Table 5 is the headline result. For each stock detector (YOLOv8n, YOLOv10n) and each arm we report navigation accuracy, calibration, detection, and edge throughput, as mean $\pm$ std over five seeds on the CRDLD test split (in-domain), with cross-dataset transfer to CRBD reported alongside. The final row, $\Delta(B-A)$ with its 95 % bootstrap confidence interval, summarizes the end-to-end effect of CALM-Row over the equal-weight baseline. The table is designed to show whether CALM-Row reduces heading and line-fit error without degrading mAP, and whether the FPS column confirms zero added on-device cost: the exported detector graph is a vanilla YOLO and CALM-Row is post-NMS CPU math, so the detector FPS for A, A', B0, and B should coincide up to measurement noise.

We additionally report mAP@50–95 with its across-seed variance alongside Table 5 (omitted from the main table for width). Because CALM-Row adds no learnable detector parameters and its losses act only on the existing DFL distribution, the expectation is that detection accuracy is unchanged within noise while navigation accuracy improves; this expectation will be confirmed or refuted by the completed runs.

Table 5: T1 — Main A/B comparison on CRDLD test, mean $\pm$ std over 5 seeds. Heading error and line-fit RMSE are primary; cross-track is a static geometric proxy at the look-ahead row (Section 4.5); FPS is detector-only INT8 throughput on the named edge device. ECE is undefined for the unweighted readouts A/A' (no per-box variance is consumed). *Results to be completed.*

Detector	Arm	Heading ($^{\circ}$) $\downarrow$	RMSE (px) $\downarrow$	RMSE (cm) $\downarrow$	Cross-track (cm) $\downarrow$	ECE $\downarrow$	mAP@50	FPS
YOLOv8n	A equal-LS	—	—	—	—	n/a	—	—
	A' RANSAC	—	—	—	—	n/a	—	—
	B0 weighted (raw var)	—	—	—	—	—	—	—
	B CALM-Row	—	—	—	—	—	—	—
	Δ (B–A) [95 % CI]	—	—	—	—	—	—	—
YOLOv10n	A equal-LS	—	—	—	—	n/a	—	—
	A' RANSAC	—	—	—	—	n/a	—	—
	B0 weighted (raw var)	—	—	—	—	—	—	—
	B CALM-Row	—	—	—	—	—	—	—
	Δ (B–A) [95 % CI]	—	—	—	—	—	—	—

5.2. Paired significance tests (T2)

Because A, A', and B0 are evaluated on the identical checkpoint and B is paired per frame against them, the comparisons are tested with the Wilcoxon signed-rank test on per-frame heading error (with 5 seeds, normality cannot be assumed for a t -test). Table 6 reports the median paired difference, the Holm–Bonferroni-adjusted p -value across the confirmatory family, and the Cliff's δ effect size. The pre-registered primary hypothesis H1 (CALM-Row beats equal-weight LS on heading error) passes only if the adjusted $p < 0.05$ and $|\delta| \geq 0.33$, i.e. the effect must be both significant and non-trivial. The two decomposition rows (B0 vs. A; B vs. B0) are reported regardless of outcome so that the source of any gain — free weighting versus calibrated training — is stated plainly. A far-field-focused replicate of Table 6, restricted to the genuinely-far held-out rows (top-of-frame, smallest boxes; the de-circularized far-field test of Section 4.6), is reported separately. The pre-registered expectation is that the CALM-Row advantage is *larger* in this stratum, because the far boxes are where the DFL distribution is flattest (largest σ_e^2 , hence largest centroid variance $\sigma_{c_x}^2$),

Table 6: T2 — Paired significance on heading error (primary metric), per detector, CRDLD test. Wilcoxon signed-rank, Holm-adjusted across the confirmatory family; Cliff’s δ as effect size. H1 passes iff adjusted $p < 0.05$ and $|\delta| \geq 0.33$. *Results to be completed.*

Detector	Comparison	median Δ (°)	Wilcoxon p (Holm-adj)	Cliff’s δ
YOLOv8n	B vs. A (full effect)	—	—	—
	B0 vs. A (weighting only)	—	—	—
	B vs. B0 (training adds)	—	—	—
YOLOv10n	B vs. A (full effect)	—	—	—
	B0 vs. A (weighting only)	—	—	—
	B vs. B0 (training adds)	—	—	—

where the precision weights w_i down-weight most aggressively, and where heading is most sensitive. This is the mechanism the method is designed to exploit, measured here on real, un-manipulated frames with mask-derived ground-truth lines rather than on injected noise.

5.3. Auxiliary-loss ablation (T3)

Table 7 attributes the trained gain (B over B0) to the two auxiliary losses by switching one off at a time: **B–cal** (L_{DDC} off, L_{LSL} on) and **B–line** (L_{LSL} off, L_{DDC} on), each compared against full B and against the readout-only B0. The pre-registered expectation is that L_{DDC} chiefly drives calibration (ECE), L_{LSL} chiefly drives heading/line-fit accuracy by letting gradients flow through the precision weights w_i into the DFL logits, and that both are needed for the full effect. Ablations use three seeds (stated), fewer than the five used for the confirmatory tables, and are therefore reported as indicative.

Table 7: T3 — Auxiliary-loss ablation (YOLOv8n, CRDLD test, 3 seeds). L_{DDC} = distance-aware calibration; L_{LSL} = line-stability. *Results to be completed.*

Variant	L_{DDC}	L_{LSL}	Heading ($^{\circ}$)↓	RMSE (px)↓	ECE↓
B0 (readout only)	✗	✗	—	—	—
B-cal	✗	✓	—	—	—
B-line	✓	✗	—	—	—
B (full CALM-Row)	✓	✓	—	—	—

5.4. Neck-agnosticity (T4)

Because CALM-Row is post-head and reads only the existing DFL distribution, it should be indifferent to the detector’s feature-fusion neck. Table 8 repeats the A-vs-B heading comparison across published necks — stock PAN [19], ASFF [21], and BiFPN [20] — to test whether the gain is neck-specific (pre-registered hypothesis H3). Each row reports the equal-weight (A) and CALM-Row (B) heading error and their difference; the pre-registered expectation is a negative, roughly consistent Δ across all necks. These necks are used only as published baselines to probe agnosticity; this paper makes no neck-architecture novelty claim.

Table 8: T4 — Neck-agnosticity (YOLOv8n backbone, CRDLD test). A-vs-B heading error on published necks; H3 expects a consistent negative Δ . *Results to be completed.*

Neck	A heading ($^{\circ}$)	B heading ($^{\circ}$)	Δ (B–A)↓
Stock PAN [19]	—	—	—
ASFF [21]	—	—	—
BiFPN [20]	—	—	—

5.5. Edge latency and quantization (T5)

Table 9 reports deployment efficiency on the two named edge stacks: TensorRT (FP16 and INT8) on the Jetson device and RKNN INT8 on the Rockchip board. We report mean $\pm$ std and P95 latency, throughput, power, and energy per frame, together with the CALM-Row post-NMS CPU overhead measured separately. The central deployment claim is verifiable here: since CALM-Row introduces zero new NPU operators, the exported INT8 graph for arms A, A', B0, and B is identical, so the NPU latency columns should coincide across arms and the only CALM-Row cost is the small CPU line-fit. We also report ECE before and after INT8 quantization (re-fitting the optional inference-time calibrator on a small validation set post-INT8) to support the claim that calibration survives quantization.

Table 9: T5 — Edge latency, throughput, and power. TensorRT on Jetson; RKNN INT8 on Rockchip. CALM-Row overhead is post-NMS CPU math (no NPU ops). Report at the stated input resolution, batch size 1, after warm-up. *Results to be completed.*

Device / runtime	Precision	Latency mean $\pm$ std (ms)	P95 (ms)	FPS	Power (W)	Energy/frame (mJ)
Jetson / TensorRT	FP16	—	—	—	—	—
Jetson / TensorRT	INT8	—	—	—	—	—
Rockchip / RKNN	INT8	—	—	—	—	—
CALM-Row post-NMS CPU overhead		—	—	—	—	—

5.6. Synthetic mechanism illustration (not a dataset result)

To make the precision-weighting mechanism concrete *independently* of any dataset, the reference implementation includes a small synthetic check (a controlled `numpy/torch` demonstration, not an experiment on real imagery). Box-edge logits are generated so that near boxes have sharp DFL distributions (low variance σ_e^2) and far boxes have flat distributions (high variance); the far centroids are then corrupted with injected lateral noise. On this fixed synthetic configuration, fitting

the guidance line $x = ay + b$ with equal weights yields an *illustrative* heading error of 1.16° , whereas the precision-weighted GLS fit with $w_i = 1/\sigma_{c_x,i}^2$ yields 0.65° on the same points. These numbers are reported purely as an **illustration of the mechanism** and are *not* a dataset result: they show that down-weighting high-variance far boxes recovers a more accurate heading when the variance genuinely tracks reliability. The corresponding real, de-circularized evidence is the far-field analysis accompanying Table 6, measured on un-manipulated CRDLD frames with mask-derived ground-truth lines.

5.7. Summary of planned findings

Taken together, the tables above are designed to answer four questions, each tied to a core evaluation requirement. (i) *Does the application improve?* Tables 5–6 measure navigation directly in heading (deg) and cross-track (cm), not detection mAP. (ii) *Is the comparison against real prior art?* Arms A/A' reproduce published line-fitting recipes [4], and the system-level positioning (reported separately) places CALM-Row against crop-row pipelines [2, 5, 6] on the same split, GT line, and device. (iii) *Is the analysis rigorous?* Multiple seeds, per-frame paired Wilcoxon tests with Holm correction, effect sizes with bootstrap CIs, and a separated across-seed/across-field variance budget are all pre-registered. (iv) *Is the uncertainty real rather than a relabelled “depth”?* Calibration is measured (ECE, reliability) against *realised* localization error and tested for survival under INT8, using only the existing DFL distribution [1] rather than a separate uncertainty head [7, 8]. All cells will be populated once the runs in the experiment plan complete.

6. Discussion

6.1. Why calibrated far-field uncertainty should reduce heading error

The guidance line in a detect-then-fit pipeline is anchored by the same detections that are least reliable. Under a forward-facing field camera, rows near the top of the image span few pixels, are partially occluded by foreground canopy, and project many ground metres into a single image row; the detector’s box edges there are correspondingly diffuse. In our parameterisation $x = ay + b$, the heading $\theta = \text{atan}(a)$ is the controller-relevant readout, and the slope a is governed disproportionately by the spread of the centroids in y . A handful of far-field centroids that are displaced by tens of pixels can therefore swing θ by several degrees even when every near-field box is perfectly placed — which is precisely the regime an equal-weight least-squares fit cannot defend against, because it treats a noisy far box and a sharp near box as equally informative.

CALM-Row addresses this at the only point where the information is still available: the distributional DFL head [1] already emits, per box edge $e \in \{l, t, r, b\}$, a softmax distribution $p_e(k)$ over $R = \text{reg_max}$ bins, whose variance $\sigma_e^2 = \sum_k (k - \mu_e)^2 p_e(k)$ widens exactly when the edge is uncertain. By linear error propagation through the centroid, the per-box horizontal variance is $\sigma_{c_x}^2 = (s^2/4)(\sigma_l^2 + \sigma_r^2)$, and the precision weight $w_i = 1/\sigma_{c_x,i}^2$ down-weights diffuse far boxes inside the GLS normal equations. Because the weighting acts multiplicatively on each residual, its leverage is largest exactly where $\sigma_{c_x}^2$ is largest — the far field — and negligible where the detector is confident. This is the mechanistic reason we expect the largest A/B gap to appear on the held-out far rows rather than on the full-frame average, and it is why the pre-registered far-field analysis (Section 5) is the discriminating test rather than the headline mean. An illustrative synthetic check (Section 5.6) only confirms that the mechanism behaves as derived; the empirical claim rests entirely

on the real, un-manipulated far-field test.

Crucially, the weights are not free parameters to be tuned but a *calibrated* quantity. The distance-aware calibration loss (DDC) is a Gaussian negative-log-likelihood that ties $\sigma_{c_x}^2$ to the realised squared centroid error on matched anchors, so a predicted variance of a given magnitude must, on validation data, correspond to localisation errors of the matching magnitude. This is what lets us report reliability (ECE, coverage of realised heading error at 68/95%) rather than assert that the network has learned “depth”; the uncertainty is a measured, checkable property (reported as ECE and coverage).

6.2. Decomposing the gain: weighting alone (B0) versus training (B)

A reviewer’s natural question is whether any improvement comes from the *idea* of uncertainty weighting or from the *extra training*. Our design answers this by construction. Arm B0 reads out a precision-weighted GLS line from the raw DFL variance of a *stock* detector trained with standard losses — it shares one checkpoint with the equal-weight baseline A and the RANSAC baseline A’, so the three are compared paired per frame with zero training-induced variance between them. Arm B is a separate checkpoint trained with the DDC and LSL auxiliary losses. The contrast B0 versus A isolates the value of the *free* variance the head already produces; the contrast B versus B0 isolates what the calibration-and-line-aware training *adds* on top. We report both regardless of which dominates: if B0 already beats A, the result is a near-zero-cost readout change requiring no retraining; if the gain only materialises in B, the training is doing the work and we say so plainly. Either outcome is an honest, useful finding, and the decomposition forecloses the “the gain is just more compute” critique.

The expected qualitative pattern follows from the two losses. Raw DFL variance is correlated with localisation error but is not guaranteed to be *calibrated* in magnitude, so B0 should capture the

gross near/far ordering but may mis-scale the weights; DDC re-scales the variance to match realised error, and LSL — whose gradients flow through w_i into the DFL logits, with the far look-ahead row explicitly included in its RMSE term — additionally teaches the network to be *decisive about which far boxes to distrust*, pushing unreliable far edges toward flatter distributions. We therefore expect $B_0 \leq B$ on the far-field heading metric, but we treat the size of the $B-B_0$ increment as an empirical question to be filled from the results (Section 5) rather than asserted.

6.3. What CALM-Row is, and is not

We are deliberate about the scope of the contribution. CALM-Row is *not* an architectural novelty: it introduces no new neck, no new backbone block, and — in its training losses — no new learnable parameters, since those losses are computed entirely from the variance the DFL head [1] already produces. The detector is a stock YOLOv8n / YOLOv10n, and the exported INT8/RKNN or TensorRT graph is a vanilla YOLO; CALM-Row runs as post-NMS CPU arithmetic and adds zero NPU operators. This is a feature, not a hedge: it means any improvement is attributable to the estimator and the navigation-task training, not to an unaccounted increase in capacity, and it cleanly separates our claim from feature-fusion necks such as ASFF [21], which improve the features a detector computes rather than the line fit.

Positioned honestly, CALM-Row is a *navigation-native, calibrated-uncertainty estimator* that sits between a detector and a controller. Detection-uncertainty methods such as KL-Loss [8] and Gaussian-YOLO [7] model box uncertainty to improve *detection*; we instead read a calibrated per-detection centroid covariance out of the existing DFL distribution and propagate it into the downstream geometric estimate that the application actually consumes. Relative to crop-row pipelines — equal-weight least-squares centerlines on YOLOX maize rows [4], segmentation-plus-scan methods on the CRDLD benchmark [2, 3], polynomial regressors [6], and row-anchor

regressors ported from lane detection [5] — the differentiator is not a new output paradigm but that the line is fit with *principled, calibrated weights* and is trained and evaluated in navigation units (heading in degrees, cross-track in centimetres) rather than detection mAP. We make no claim that CALM-Row’s detector is more accurate than these systems in mAP; the claim is confined to the guidance-line readout and its calibration, evaluated under a controlled A/B in which only the line-fit axis varies.

6.4. Limitations

We state the boundaries of the evidence explicitly, since several of them bound how far the conclusions generalise.

Generalisation and its limits. Generalisation is assessed in two ways: in-domain on the held-out CRDLD test split, and as *cross-dataset* transfer to CRBD [23] (a different camera, crop and resolution), evaluated without retraining. The CRDLD download used here carries no per-field metadata, so a within-dataset leave-one-field-out protocol is unavailable; the cross-dataset CRBD transfer takes its place and is, if anything, a stronger probe. The unit of generalisation is the dataset, not the seed.

Flat-ground homography for metric units. Centimetre-scale metrics require an image-to-ground homography, which we recover under a *flat-ground* assumption (or anchor to known inter-row spacing where intrinsics are unavailable). On sloped or uneven terrain this assumption is violated, and a non-zero camera pitch inflates the metric error most at the far look-ahead distance — the same region CALM-Row targets. We therefore report degrees (which are controller-independent and free of the homography) as the primary metric, give centimetre figures only where the calibration is

662 defensible, and provide an explicit pitch-to-centimetre error budget so the metric claims are not
663 over-read.

664 **Cross-track is a static geometric proxy, not a closed-loop result.** We report cross-track as the
665 lateral offset (px, and cm where the homography permits) between the predicted and ground-truth
666 guidance lines at the look-ahead row. We deliberately do *not* run a closed-loop pure-pursuit
667 simulation and do *not* report an intervention rate: both require sequential field video and a controller
668 model and are left to future work. The controller-independent heading error is our primary navigation
669 metric, and a real-robot validation likewise remains future work.

670 **Dependence on the DFL spread.** CALM-Row’s weights are only as informative as the variance
671 the DFL head carries. If $R = \text{reg_max}$ is too small, or the head collapses toward near-deterministic
672 distributions, the per-edge variance saturates and the weights lose discriminative power between
673 near and far boxes; the method would then degenerate toward equal-weight least-squares. Stock
674 Ultralytics uses $R = 16$ for all model sizes, which we expect to carry sufficient spread, and we
675 ablate $R \in \{8, 16\}$ to test this dependence rather than assume it. The approach also presupposes a
676 distributional regression head; it does not transfer unchanged to detectors with a single-point box
677 regressor.

678 **Calibration after INT8 quantisation.** The variance the head emits can shift under INT8
679 quantisation, so calibration established in floating point need not survive on the deployed graph. We
680 therefore re-fit the optional post-hoc CalibScale affine on a small validation set after quantisation
681 and report calibration before versus after INT8. We treat “calibration survives quantisation” as a
682 hypothesis to be demonstrated, not assumed; if the post-INT8 reliability degrades, the deployment-
683 time recommendation is to re-calibrate, and the weighting — being monotone in the variance —

still preserves the near/far ordering even when the absolute scale drifts.

Ground-truth line extraction choices. The evaluation depends on a ground-truth guidance line, and its definition is a modelling choice. For evaluation we use a *mask-derived* centerline (a frozen row-wise robust regression, Section 4.3) that is independent of the detector, deliberately avoiding the circularity of fitting the ground-truth line through ground-truth box centroids when the very claim under test is weighted-versus-unweighted fitting through box centroids. We cross-check the automatically extracted line against a human-annotated subset and report inter-annotator agreement in degrees and centimetres. Residual sensitivity to the extraction algorithm — gap and occlusion handling, the choice of the navigation row, and the robust-regression estimator — is a limitation of any line-level benchmark and bounds the resolution at which differences smaller than the annotation agreement can be trusted. For training, the line-stability loss instead uses a line fit through *ground-truth box centroids*, which is detector-independent and hence non-circular with respect to the evaluation; the two definitions serve different purposes and are reported separately.

7. Conclusions

Vision-based crop-row guidance is dominated by a single recipe: a detector localises row segments as bounding boxes, a guidance line is fit through the box centroids, and the line is handed to a steering controller [2, 4]. Two gaps persist across this literature and across the alternative segmentation, row-anchor and curve-regression paradigms [2, 5, 6]. First, the systems are routinely tuned and reported in detection units (mAP), whereas the deployed quantity is a *navigation* error—heading in degrees and cross-track distance in centimetres—and the two are not interchangeable. Second, regardless of paradigm, the guidance line is fit with equal or heuristic point weights, even though the far-field detections that anchor the look-ahead heading are precisely the least reliable points in the

frame. Box-regression uncertainty has been modelled before, but for better *detection* (KL-Loss [8], Gaussian-YOLO [7]) rather than read out as a calibrated per-detection covariance and propagated into the line fit.

This paper has addressed both gaps with CALM-Row, a post-head module for a stock lightweight detector. CALM-Row exploits an uncertainty signal that the detector already produces for free: the Distribution Focal Loss (DFL) head [1] emits, per anchor, a discrete probability distribution $p_e(k)$ over R bins for each of the four box edges $e \in \{l, t, r, b\}$. We read the variance $\sigma_e^2 = \sum_k (k - \mu_e)^2 p_e(k)$ of that existing distribution, propagate it by linear error propagation into a per-box centroid variance, $\sigma_{c_x}^2 = (s^2/4)(\sigma_l^2 + \sigma_r^2)$ (with stride s), and use the resulting precision $w_i = 1/\sigma_{c_x,i}^2$ to weight a differentiable generalized-least-squares (GLS) fit of the guidance line $x = a y + b$. The heading $\theta = \arctan a$ and its uncertainty follow in closed form from the weighted parameter covariance. The detector is trained with two auxiliary losses computed entirely from this distribution and from matched ground-truth centroids, and introducing no new learnable parameters: a distance-aware calibration loss (DDC), a Gaussian negative-log-likelihood that ties the predicted centroid variance to the realised squared centroid error so that the variance becomes a *calibrated* uncertainty; and a line-stability loss (LSL), whose gradients flow through the weights w_i into the DFL logits so the network learns to down-weight unreliable far boxes. The ground-truth line used in LSL is fit (unweighted) through ground-truth box centroids and is therefore detector-independent, while the evaluation line is mask-derived and independent, so the central weighted-versus-unweighted claim is not evaluated circularly.

CALM-Row is deliberately *not* an architectural contribution. It introduces no new operator on the feature maps and no new learnable parameters in the training losses; all computation is post-head, CPU-side arithmetic over post-NMS boxes. The exported INT8/RKNN (Rockchip) or TensorRT (Jetson) graph is therefore a vanilla YOLO with zero added NPU operations, and because the module

reads only the standard DFL distribution it is detector-agnostic, applying without modification to YOLOv8n, YOLOv10n and any DFL-headed successor. A synthetic mechanism check, clearly labelled as illustrative, is reported in Section 5.6.

We have specified, rather than yet executed, an evaluation designed around the deployed task. The primary metric is controller-independent heading error in degrees, with line-fit RMSE (px and cm via a stated ground-plane homography) and a static geometric cross-track proxy in centimetres as secondary measures (a closed-loop pure-pursuit simulation is left to future work); calibration is reported via expected calibration error and reliability, before and after INT8 quantization. The core comparison is a paired, per-frame A/B between equal-weight least squares (the de-facto recipe), a readout-only arm that applies the raw DFL variance with no extra training, and the full CALM-Row, isolating the gain from weighting alone versus from calibration-aware training. Generalization is assessed in-domain on the CRDLD [2] test split and as cross-dataset transfer to CRBD [23] (different camera, crop and resolution) without retraining, with reliability stratified continuously against an image-row range proxy rather than categorical distance bins. All confirmatory comparisons use multiple seeds, paired non-parametric tests with multiple-comparison correction, and effect sizes with bootstrap confidence intervals; published crop-row systems [4–6] are positioned on the same test split, ground-truth line and edge device. We deliberately make no quantitative claims here: all navigation, calibration and efficiency numbers remain to be measured under this protocol [TODO: report measured values once experiments are run].

Several directions extend this work. The most important is real closed-loop validation: replacing the simulator with field trials on the named edge platforms, where measured perception latency, controller dynamics and terrain interact with the guidance estimate. A second direction is temporal consistency—fusing per-frame line estimates and their calibrated heading variance across time, for example with an uncertainty-aware filter, to suppress frame-to-frame jitter at the look-ahead

row. A third is learned geometric range supervision: where stereo, LiDAR or known inter-row spacing is available, the currently free, monocular uncertainty could be tied to true metric range and ground-plane geometry, turning the calibrated covariance from an image-space proxy into a metrically grounded reliability estimate without compromising the zero-NPU-cost, detector-agnostic deployment that motivates CALM-Row.

Acknowledgments

This work has been supported by VNU University of Engineering and Technology under project number CN24.20.

Contributions

Conceptualization, investigation, and formal analysis: Manh V. Hoang, Nam Do and Thang M. Pham; methodology, validation: Manh V. Hoang and Thang M. Pham; software and writing: Manh V. Hoang. All authors were involved in its revision. All authors read and approved the final manuscript.

References

- [1] Xiang Li et al. “Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020, pp. 21002–21012.
- [2] Rajitha De Silva et al. “Deep learning-based crop row detection for infield navigation of agri-robots”. In: *Journal of field robotics* 41.7 (2024), pp. 2299–2321.
- [3] Rajitha De Silva, Grzegorz Cielniak, and Junfeng Gao. “Vision based crop row navigation under varying field conditions in arable fields”. In: *Computers and Electronics in Agriculture* 217 (2024), p. 108581.
- [4] Hailiang Gong, Weidong Zhuang, and Xi Wang. “Improving the maize crop row navigation line recognition method of YOLOX”. In: *Frontiers in Plant Science* 15 (2024), p. 1338228. doi: 10.3389/fpls.2024.1338228.
- [5] Bofeng Feng et al. “ALNet: towards real-time and accurate maize row detection via anchor-line network”. In: *Frontiers in Plant Science* 16 (2025), p. 1706596. doi: 10.3389/fpls.2025.1706596.
- [6] Rahul Harsha Cheppally and Ajay Sharda. “RowDettr: End-to-End Crop Row Detection Using Polynomials”. In: *arXiv preprint arXiv:2412.10525* (2024).
- [7] Jiwoong Choi et al. “Gaussian YOLOv3: An Accurate and Fast Object Detector Using Localization Uncertainty for Autonomous Driving”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, pp. 502–511.
- [8] Yihui He et al. “Bounding Box Regression with Uncertainty for Accurate Object Detection”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 2888–2897.
- [9] Xiang Li et al. “Generalized Focal Loss V2: Learning Reliable Localization Quality Estimation for Dense Object Detection”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021, pp. 11632–11641.
- [10] Zequn Qin, Huanyu Wang, and Xi Li. “Ultra Fast Structure-aware Deep Lane Detection”. In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2020, pp. 276–291.
- [11] Lucas Tabelini et al. “Keep your Eyes on the Lane: Real-time Attention-guided Lane Detection”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021, pp. 294–302.
- [12] Boliao Li et al. “Rethinking the crop row detection pipeline: An end-to-end method for crop row detection based on row–column attention”. In: *Computers and Electronics in Agriculture* 225 (2024), p. 109264. doi: 10.1016/j.compag.2024.109264.

-
- [13] Henning T Søgaaard and Henrik J Olsen. “Determination of crop rows by image analysis without segmentation”. In: *Computers and Electronics in Agriculture* 38.2 (2003), pp. 141–158.
- [14] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. *Ultralytics YOLOv8*. Ultralytics GitHub repository, accessed April 2, 2026. 2023. URL: <https://github.com/ultralytics/ultralytics>.
- [15] Chien-Yao Wang, I-Hau Yeh, and Hong-Yuan Mark Liao. “Yolov9: Learning what you want to learn using programmable gradient information”. In: *European conference on computer vision*. Springer. 2024, pp. 1–21.
- [16] Ao Wang et al. “YOLOv10: Real-Time End-to-End Object Detection”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2024.
- [17] Wouter Van Gansbeke et al. “End-to-end Lane Detection through Differentiable Least-Squares Fitting”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*. 2019.
- [18] Tsung-Yi Lin et al. “Feature pyramid networks for object detection”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 2117–2125.
- [19] Shu Liu et al. “Path aggregation network for instance segmentation”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 8759–8768.
- [20] Mingxing Tan, Ruoming Pang, and Quoc V Le. “Efficientdet: Scalable and efficient object detection”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020, pp. 10781–10790.
- [21] Songtao Liu, Di Huang, and Yunhong Wang. “Learning Spatial Fusion for Single-Shot Object Detection”. In: *arXiv preprint arXiv:1911.09516* (2019).
- [22] Alex Kendall and Yarin Gal. “What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision?” In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017.
- [23] Ivan Vidović, Robert Cupec, and Željko Hocenski. “Crop row detection by global energy minimization”. In: *Pattern Recognition* 55 (2016), pp. 68–86.
- [24] Gongfan Fang et al. “Depgraph: Towards any structural pruning”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2023, pp. 16091–16101.